Reviewer Pack — Minimal Tropical Root Separation in Max-Plus Semiring vs AC0…

Entry efb96a4bb68e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 09:41:58 UTC

Minimal Tropical Root Separation in Max-Plus Semiring vs AC0 Circuit Lower Bounds

Entry ID: `efb96a4bb68e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 09:41:58 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry over Max-Plus Semiring **Field B** (complexity object): AC0 Circuit Complexity

Statement:

> {'s1': 'For any max-plus semiring tropical polynomial f, the minimal separation between distinct roots in its maximal ideal is lower bounded by a function of its degree.', 's2': 'This separation translates to a lower bound on the size of an AC0 circuit computing the same function over {0, 1}.', 's3': 'Consequently, there exists a constant c such that for all tropical polynomials f, min_root_separation(f) >= c * degree(f)^d for some d > 0.'}

Rationale (proposer's reasoning):

> {'s1': 'Tropical geometry provides an algebraic framework to study functions on the real line over the max-plus semiring.', 's2': 'The separation of roots in a maximal ideal can be related to circuit complexity by considering the minimal number of inputs needed to distinguish between function values.', 's3': 'This connection could potentially lift lower bounds from AC0 circuit complexity to tropical geometry, and vice versa.'}

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2a25353a336af53d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For tropical polynomials f, if min_root_separation(f) >= c * degree(f)^d and all AC0 circuits computing f have size <= c' * degree(f)^e for some constants c, c', d, e with d > 0, the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry over max-plus semiring" AND "AC0 circuit complexity" - "max-plus semiring tropical polynomial" AND min_root_separation - "degree lower bound" AND AC0 circuit size AND tropical geometry max-plus

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import itertools
import math

def max_plus_polynomial(degree, coefficients):
    return [coefficients[i] + i for i in range(degree)]

def evaluate_poly(poly, x):
    result = 0
    for coeff in poly:
        result = max(result, coeff + x)
    return result

def min_root_separation(poly):
    roots = []
    degree = len(poly) - 1
    if degree == 0:
        return float('inf')

    # Use a simple root-finding method (e.g., bisection)
    low, high = -100, 100
    while low < high:
        mid = (low + high) / 2
        value = evaluate_poly(poly, mid)
        if value == degree * mid:
            roots.append(mid)
            break
        elif value > degree * mid:
            high = mid
        else:
```

```
            low = mid

    return min(abs(r1 - r2) for r1, r2 in itertools.combinations(roots, 2)) if roots else float('i

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    degree = n
    coefficients = [random.randint(-10, 10) for _ in range(degree + 1)]
    poly = max_plus_polynomial(degree, coefficients)

    min_separation = min_root_separation(poly)
    if min_separation == float('inf'):
        return {
            "metric_name": "min_root_separation",
            "metric_value": min_separation,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "No distinct roots found"
        }

    # Placeholder for AC0 circuit complexity calculation
    # This is a stub and should be replaced with actual implementation
    circuit_size = degree ** 2

    return {
        "metric_name": "min_root_separation",
        "metric_value": min_separation,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "AC0 circuit complexity calculation not implemented"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
    std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value) ** 2 for res in res
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"AC0 circuit complexity calculation not implemen
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
c_value': inf, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'No distinct roots
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
TRIAL: {'metric_name': 'min_root_separation', 'metric_value': inf, 'instances_tested': 1, 'conjectur
RESULT: FALSIFIED counterexample="AC0 circuit complexity calculation not implemented" first_failing_s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test has only one instance tested, which is insufficient to draw a definitive conclusion. The metric 'min_root_separation' returned infinity for all trials, indicating that the calculation of AC0 circuit complexity was not implemented. This suggests a potential bug in the metric definition or an incomplete implementation, rather than evidence against the conjecture itself.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test failed to find distinct roots for a tropical polynomial, which contradicts the conjecture that there should be a lower bound on the separatio | next: Investigate the cause of the 'No distinct roots found' error. If it is due to an incomplete implementation or a bug in the metric definition, address these issues before re-evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14861
2	preregistration	ollama_remote	glm4:latest	0	0	10143
3	novelty	ollama_remote	glm4:latest	0	0	8323
4	novelty	ollama_remote	glm4:latest	0	0	8699
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14190
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10107
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9314
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9441

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	critic	ollama_remote	glm4:latest	0	0	13584
10	judge	ollama_remote	glm4:latest	0	0	9260

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 107922 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/efb96a4bb68e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/efb96a4bb68e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/efb96a4bb68e.tar.gz` (if generated)